

Fast Multi-Agent Path Planning with Turn Actions: A Priority Inheritance Approach (Supplementary File)

Zheyu Tao¹, Chunlong Yu^{2*}

APPENDIX A

This section provides details on the breadth-first search (BFS) in node selection strategy. To obtain the costs from all states (each orientation of a specific node represents a distinct state) to a specific state on goal nodes in a given map, we initiate BFS to complete this task in a single pass and store these costs in a cost table.

In classical MAPF, BFS is performed starting from the goal node, iteratively exploring adjacent nodes until the current node is reached. This approach assumes that moving to an adjacent node incurs a unit cost under the classical MAPF motion model. However, in MAPF with turn actions (MAPF_T), the introduction of additional turn actions means that moving to an adjacent node may no longer incur a cost of 1. Therefore, based on the action model of MAPF_T, we treat states before and after executing a unit-cost action as adjacent states and perform BFS between these adjacent states.

Since the BFS in our approach also starts from the goal state and searches backward, it employs a reverse motion model to accurately compute the cost of reaching the goal state from a specific state in the forward direction. As shown in Figure 1, there are three types of actions connecting adjacent states in BFS. (1) rotating 90 degrees clockwise, (2) rotating 90 degrees counterclockwise, and (3) moving one unit in the opposite direction to the current orientation.

When the search reaches the current state, it returns the minimum cost from the current state to the goal state. Notably, since the orientation upon reaching the goal node can be arbitrary, we select the one that results in the least cost.

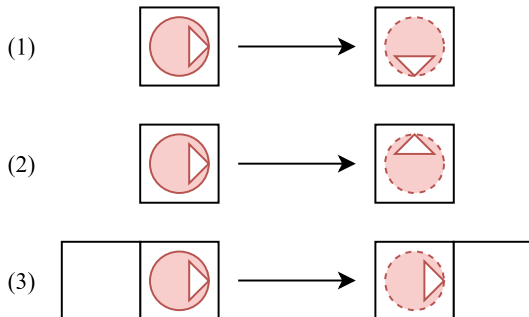


Figure 1. Three types of adjacent states in BFS

APPENDIX B

This section provides examples of failures of PIBT_T. All failures of PIBT_T in the experiments are due to reaching the predefined makespan limit. Cause of the failures can be categorized into two types.

The first type of failure occurs due to the absence of graph condition. Our approach can only ensure that all agents reach the goal at least once within finite timesteps if there is a simple cycle for all pairs of adjacent nodes in the map [1].

For instance, the map in Figure 2 does not meet this condition. In this problem, agents 1 and 2 have goals D and A, respectively, with agent 1 having a higher priority than agent 2. When executing PIBT_T for agent 1, it requests node D, which is occupied by agent 2, triggering PIBT_T for agent 2. However, agent 2 can request neither node B nor D in the next timestep without causing a conflict, forcing it to stay in place and return False. Consequently, agent 1 must request an alternative node. Its second choice is node B, its current location, as it has the lowest cost to the goal. This results in both agents wait in place in the next timestep, leading to a deadlock.

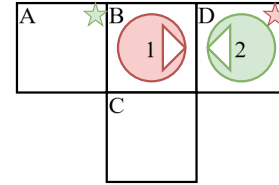


Figure 2. Deadlock caused by absence of graph condition

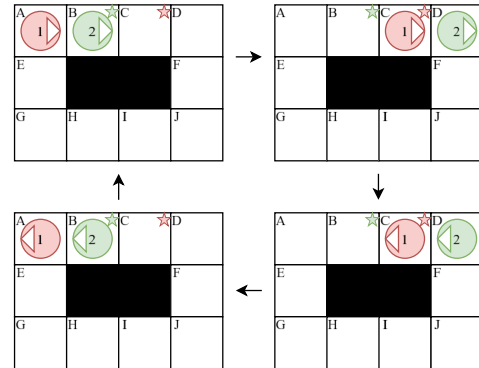


Figure 3. PIBT_T cannot ensure that all agents reach their goals simultaneously

*Research supported by National Natural Science Foundation of China (72301196) and Tongji University "Fundamental Research Funds for the Central Universities" (22120240607). (Corresponding author: Chunlong Yu)

¹Zheyu Tao is with the School of Mechanical Engineering, Tongji University, Shanghai, 201804, P.R. China.

²Chunlong Yu is with the School of Mechanical Engineering, Tongji University, Shanghai, 201804, P.R. China (e-mail: chunlong_yu@tongji.edu.cn).

The second type of failure stems from the "reachability" property of PIBT [1], which ensures that all agents reach the goal at least once within a finite number of timesteps under the graph condition, rather than simultaneously staying at the goal. In our method, after assigning initial priorities, we increment the priority value of agents that have not reached the goal at each timestep. To ensure that such agents have higher planning priority, once an agent reaches the goal, its priority is reset to the initial value. This leads to a situation where higher-priority agents that have not yet reached the goal may push away those already at the goal and be pushed after arrival conversely. As a result, two agents take turns to reach their own goal.

As illustrated in Figure 3, agents 1 and 2 have goals C and B, respectively. Suppose agent 1 is at node A, and agent 2 has just reached its goal and reset its priority. In this case, agent 1, with a higher priority, will push agent 2 away from node B until it reaches its goal. Once agent 1 reaches its goal, agent 2, whose priority gradually increases after leaving the goal, will force agent 1 to vacate its goal due to its own reset priority. This loop repeats, leading to the second type of failure.

When pushed away by a higher-priority agent, the agent will not request adjacent nodes perpendicular to its current direction, as the cost of reaching the goal through these nodes and then returning is higher than that of continuing forward and turning back.

APPENDIX C

This section provides details on complexity analysis of our proposed approach.

As shown in Algorithm 1 in the paper, within a single timestep of planning, sorting the agents based on updated priorities (line 2) requires $O(|A|\lg|A|)$. During the planning process for each agent, iterating over the candidate nodes requires $O(\Delta(G) + 1)$, where $\Delta(G)$ is the maximum degree of the graph. Both detecting cycles and handling cycles requires $O(|C|)$, where $|C|$ is the number of agents in the cycle with $|C| < |A|$, resulting in an overall complexity of $O(|A|)$. Assuming other operations are completed in constant time $O(K)$, the proposed algorithm achieves a time complexity of $O(|A| \cdot (\lg|A| + |A|\Delta(G) + K))$. It is evident that the proposed method requires only polynomial time complexity for planning within a single timestep, offering a significant computational advantage over algorithm with exponential time complexity [2].

The time complexity for creating the cost table is $O(4|A| \cdot (4|V| \cdot 3)) = O(|A| \cdot |V|)$. In large-scale instances involving extensive maps or a high number of agents, the preprocessing time will increase accordingly. However, experiments on benchmark maps have demonstrated acceptable preprocessing speeds, which is explained in the numerical experiments section.

REFERENCES

- [1] K. Okumura, M. Machida, X. Défago, and Y. Tamura, "Priority inheritance with backtracking for iterative multi-agent path finding," *Artificial Intelligence*, vol. 310, p. 103752, Sep. 2022.
- [2] Y. Zhang, D. Harabor, P. L. Bodic, and P. J. Stuckey, "Efficient Multi Agent Path Finding with Turn Actions," *Proceedings of the*